PYTHON
PROGRAMMING NOTES

Comprehensive Guide

BCA / B.Tech Students
Exam & Interview Preparation

DVI Study Materials

TABLE OF CONTENTS

1. Introduction to Python ..... pg 3
2. Python Basics (Variables, Data Types, I/O) ..... pg 3
3. Operators ..... pg 3
4. Control Statements ..... pg 3
5. Strings ..... pg 4
6. Lists, Tuples, Sets, Dictionaries ..... pg 4
7. Functions ..... pg 5
8. Modules & File Handling ..... pg 5
9. Exception Handling ..... pg 6
10. Object-Oriented Programming (OOP) ..... pg 6
11. Important Built-in Functions ..... pg 7
12. Interview & Exam Revision (Top 30 Q&A) ..... pg 7-8
13. Quick Revision Cheat Sheet ..... pg 9-10

1. INTRODUCTION TO PYTHON

Python is a high-level, interpreted, general-purpose programming language created by **Guido van Rossum** in 1991. It emphasizes code readability and simplicity.

Key Features

- Easy to Learn & Read
- Interpreted (no compilation)
- Dynamically Typed
- Object-Oriented + Functional
- Cross-platform
- Huge standard library
- Free & Open Source

Applications

- Web Dev (Django, Flask)
- Data Science & ML (NumPy, pandas)
- AI & Deep Learning
- Automation / Scripting
- Cybersecurity
- Game Dev (Pygame)
- Scientific Computing

Advantages	Disadvantages
Simple syntax, fast development	Slow execution (interpreted)
Large community & libraries	High memory usage
Versatile for many domains	Weak in mobile development
Great for prototyping	GIL limits multi-threading

QUICK SUMMARY

- Python = High-level, interpreted, dynamically typed language (1991)
- Used in AI/ML, Web Dev, Automation, Data Science
- Advantage: Simple syntax | Disadvantage: Slower than C/Java

2. PYTHON BASICS

Variables & Data Types

Variable: A container to store data values. No declaration needed.

```
x = 10         # int
name = "Dev"   # str
pi = 3.14      # float
flag = True    # bool
```

Data Type	Example	Description
int	x = 10	Whole numbers
float	x = 3.14	Decimal numbers
str	x = "hello"	Text
bool	x = True	True / False
list	x = [1,2,3]	Ordered, mutable
tuple	x = (1,2)	Ordered, immutable
dict	x = {"a":1}	Key-value pairs
set	x = {1,2,3}	Unordered, unique
NoneType	x = None	Represents null

Type Conversion

```
int('5')    -> 5
str(10)     -> '10'
float('3.14') -> 3.14
list((1,2)) -> [1,2]
```

Input & Output

```
name = input("Enter name: ")     # always returns string
print("Hello,", name)            # prints to console
```

```
print(f"Age: {age}")            # f-string formatting
```

Comments

```
# Single-line comment
""" Multi-line comment """
```

QUICK SUMMARY

- No need to declare variable type — Python is dynamically typed
- type() function returns the data type
- input() always returns string — convert with int(), float()
- f-strings (f'Hello {name}') are the preferred formatting method

3. OPERATORS

Type	Operators	Example
Arithmetic	+ - * / // % **	5 // 2 = 2, 5 % 2 = 1
Comparison	== != > < >= <=	5 > 3 → True
Logical	and or not	True and False → False
Assignment	= += -= *= /= //=	x += 5 → x = x+5
Bitwise	& | ^ ~ << >>	5 & 3 → 1
Membership	in not in	'a' in 'abc' → True
Identity	is is not	x is None → True/False

QUICK SUMMARY

- // is floor division, ** is power, % is modulus
- 'in' checks membership in sequences; 'is' checks object identity
- Operator precedence: ** > * / // % > + - > comparison > logical

4. CONTROL STATEMENTS

Conditionals

```
if x > 0:
    print('Positive')
elif x == 0:
    print('Zero')
else:
    print('Negative')
```

Loops

for loop

```
for i in range(5):
    print(i)  # 0 1 2 3 4
```

```
for char in 'Python':
    print(char)
```

```
for i in range(1,10,2):
    print(i) # 1 3 5 7 9
```

while loop

```
i = 0
while i < 5:
    print(i)
    i += 1
```

Loop Control

```
break    # exit loop
continue # skip iteration
pass     # placeholder
```

QUICK SUMMARY

- range(start, stop, step) — stop is exclusive
- break exits the loop; continue skips to next iteration
- pass is a no-op placeholder — used when a body is required syntactically
- else block in loops executes when loop completes without break

5. STRINGS

Creation, Indexing & Slicing

```
s = "Python"
s[0]      # 'P'    (positive indexing)
s[-1]     # 'n'    (negative indexing)
s[0:3]    # 'Pyt'  (slicing: start:stop:step)
s[::-1]   # 'nohtyP' (reverse)
```

Important String Methods

Method	Example	Result
upper() / lower()	"hello".upper()	"HELLO"
strip()	" hi ".strip()	"hi"
split(sep)	"a,b".split(",")	["a","b"]
join(list)	",".join(["a","b"])	"a,b"
replace(old,new)	"hi".replace("i","o")	"ho"
find(sub)	"hello".find("ll")	2
count(sub)	"hello".count("l")	2
startswith(s)	"Python".startswith("Py")	True
len(s)	len("hello")	5
in operator	"py" in "python"	True

String Formatting

```
name = "Dev"; age = 18
f"Name: {name}, Age: {age}"       # f-string (preferred)
"Hello %s, age %d" % (name, age)  # % operator
"Hello {}, age {}".format(name, age) # .format()
```

QUICK SUMMARY

- Strings are immutable — operations return new strings
- Indexing starts at 0; negative index counts from end
- Slicing: s[start:stop:step] — stop is excluded
- f-strings are fastest and most readable for formatting

6. LISTS, TUPLES, SETS & DICTIONARIES

Comparison Table

Feature	List	Tuple	Set	Dictionary
Syntax	[1,2,3]	(1,2,3)	{1,2,3}	{"a":1}
Ordered	Yes	Yes	No	Yes (3.7+)
Mutable	Yes	No	Yes	Yes
Duplicates	Allowed	Allowed	Not Allowed	Keys unique
Indexing	Yes	Yes	No	By key

List Methods

```
lst = [3,1,2]
lst.append(4)    # [3,1,2,4]
lst.insert(0,0)  # [0,3,1,2,4]
lst.remove(3)    # [0,1,2,4]
lst.pop()        # removes last
lst.sort()       # in-place sort
lst.reverse()    # in-place reverse
len(lst)         # length
lst[0:2]         # slicing
```

Dict Methods

```
d = {"name":"Dev","age":18}
d["name"]       # "Dev"
d.get("age")    # 18
d.keys()        # all keys
d.values()      # all values
d.items()       # key-value pairs
d.update({"city":"Delhi"})
"name" in d     # True
del d["age"]    # remove key
```

QUICK SUMMARY

- List: ordered, mutable, allows duplicates — most used collection
- Tuple: immutable list — used for fixed data, slightly faster
- Set: unordered, unique — useful for membership testing & deduplication
- Dict: key-value store — keys must be immutable (str, int, tuple)

7. FUNCTIONS

Regular Functions

```
def greet(name, msg='Hello'):
    return f'{msg}, {name}!'
```

```
greet('Dev')
# Hello, Dev!
greet('Dev', 'Hi')
# Hi, Dev!
```

*args & **kwargs

```
def total(*nums):
    return sum(nums)
total(1,2,3,4)  # 10
```

```
def info(**kw):
    print(kw)
info(name="Dev",age=18)
```

Lambda Functions

```
square = lambda x: x**2
square(5)   # 25
```

```
add = lambda a,b: a+b
add(3,4)    # 7
```

Recursion

```
def factorial(n):
    if n == 0: return 1
    return n * factorial(n-1)
```

```
factorial(5)  # 120
```

Common Mistake

Forgetting return statement

No base case in recursion → infinite loop

Mutable default args: use None not []

QUICK SUMMARY

- def keyword defines a function; return sends back a value
- Default parameters go after required parameters
- *args collects positional args as tuple; **kwargs as dict
- Lambda = anonymous single-expression function
- Recursion must have a base case to avoid infinite loop

8. MODULES & FILE HANDLING

Modules

Import Syntax

```
import math
math.sqrt(16)      # 4.0
```

```
from math import pi, sqrt
sqrt(25)           # 5.0
```

```
import numpy as np  # alias
```

Useful Built-in Modules

```
math   — sqrt, ceil, floor, pi
random — randint, choice, shuffle
os     — path, getcwd, listdir
sys    — argv, exit, path
datetime — date, time, now()
re     — regex operations
```

File Handling

```
# Writing to file
with open("f.txt","w") as f:
    f.write("Hello\n")
```

```
# Reading file
with open("f.txt","r") as f:
    data = f.read()
```

```
# Append to file
with open("f.txt","a") as f:
    f.write("World\n")
```

```
# Read lines
with open("f.txt") as f:
    for line in f:
        print(line.strip())
```

Mode	Description
"r"	Read (default) — file must exist
"w"	Write — creates/overwrites file
"a"	Append — adds to end of file
"r+"	Read and Write
"rb"/"wb"	Binary read/write

QUICK SUMMARY

- Always use with statement — it auto-closes the file

- open() modes: r(read) w(write) a(append) r+(read+write)
- Use import module or from module import name
- Create custom module: save as .py file, import by filename

9. EXCEPTION HANDLING

Syntax

```
try:
    x = int(input('Num: '))
    result = 10 / x
except ValueError:
    print('Not a number!')
except ZeroDivisionError:
    print('Cannot divide by 0')
except Exception as e:
    print(f'Error: {e}')
else:
    print('Success:', result)
finally:
    print('Always runs')
```

Common Exceptions

```
ValueError      — wrong value type
TypeError       — wrong data type
ZeroDivisionError — divide by 0
IndexError      — index out of range
KeyError        — missing dict key
FileNotFoundError — file missing
NameError       — undefined variable
AttributeError  — no such attribute
```

raise Statement

```
def check_age(age):
    if age < 0:
        raise ValueError("Negative!")
    return age
```

QUICK SUMMARY

- try: risky code | except: handle error | else: no error | finally: always
- Catch specific exceptions first, then generic Exception
- raise manually triggers an exception
- finally block runs even if return is called in try/except

10. OBJECT-ORIENTED PROGRAMMING (OOP)

Core Concepts

Class & Object

```
class Student:
    school = 'GGSIPU'  # class var

    def __init__(self, name, age):
        self.name = name   # instance
        self.age = age

    def greet(self):
        return f"Hi, I am {self.name}"

s1 = Student('Dev', 18)
print(s1.greet())
```

Pillars of OOP

Encapsulation

Bundling data + methods in a class
Private: __var (name mangled)
Protected: _var (convention only)

Polymorphism

Same method name, different behavior
Method Overriding in child class
duck typing: if it quacks like a duck...

Abstraction

Hide complex implementation
from abc import ABC, abstractmethod
Abstract class cannot be instantiated

Special Methods

```
__init__  — constructor
__str__   — string representation
__len__   — len(obj)
__repr__  — developer repr
```

Inheritance

```
class Animal:
    def speak(self):
        return 'Sound'

class Dog(Animal):
    def speak(self):
        return 'Woof'
```

```
d = Dog()
d.speak()  #
'Woof' —
overridden
```

QUICK SUMMARY

- Class = blueprint; Object = instance of class
- __init__ is the constructor, called automatically when object is created
- Inheritance: child class inherits from parent using class Child(Parent)
- 4 Pillars: Encapsulation | Inheritance | Polymorphism | Abstraction
- super() calls parent class method from child class

11. IMPORTANT BUILT-IN FUNCTIONS

Function	Syntax	Example	Output
len()	len(seq)	len([1,2,3])	3
type()	type(obj)	type(3.14)	<class 'float'>
range()	range(s,e,step)	list(range(0,6,2))	[0,2,4]
input()	input(prompt)	input("Name:")	string
print()	print(*args,sep,end)	print(1,2,sep=",")	1,2
int/float/str()	int(x)	int('42')	42
max()/min()	max(iterable)	max([3,1,4])	4
sum()	sum(iterable)	sum([1,2,3])	6
sorted()	sorted(iter,reverse)	sorted([3,1,2])	[1,2,3]
enumerate()	enumerate(iter)	list(enumerate(['a','b']))	[(0,'a'),(1,'b')]
zip()	zip(*iterables)	list(zip([1,2],[3,4]))	[(1,3),(2,4)]
map()	map(func, iter)	list(map(str,[1,2]))	['1','2']
filter()	filter(func, iter)	list(filter(bool,[0,1,2]))	[1,2]
abs()	abs(x)	abs(-5)	5
round()	round(x,n)	round(3.14159,2)	3.14

QUICK SUMMARY

- enumerate(list) gives (index, value) pairs — avoids manual counter
- zip() combines two iterables element-wise
- map() applies a function to each element of a sequence
- sorted() returns new list; list.sort() modifies in-place

12. PYTHON INTERVIEW & EXAM REVISION — TOP 30 Q&A

Q1	**What is Python?**	High-level, interpreted, dynamically typed general-purpose language.
Q2	**Is Python compiled or interpreted?**	Interpreted. Code is executed line by line.
Q3	**What is PEP 8?**	Python's official style guide for writing clean, readable code.
Q4	**Difference between list and tuple?**	List is mutable (changeable); tuple is immutable (fixed). Tuple is faster.
Q5	**What is a dictionary?**	An unordered collection of key-value pairs. Keys must be unique and immutable.
Q6	**What are *args and **kwargs?**	*args = variable positional arguments (tuple). **kwargs = variable keyword arguments (dict).
Q7	**What is a lambda function?**	Anonymous single-expression function. Syntax: lambda args: expression
Q8	**Explain list comprehension.**	[expr for item in iterable if condition] — creates list in one line.
Q9	**What is the difference between == and is?**	== compares values; is compares object identity (memory address).
Q10	**What is __init__?**	Constructor method called automatically when an object is created.
Q11	**What is self?**	Reference to the current instance of the class. Must be first param of instance methods.
Q12	**What is inheritance?**	Child class acquires properties and methods of parent class using class Child(Parent).
Q13	**What is polymorphism?**	Same method name, different behavior in different classes (method overriding).
Q14	**What is encapsulation?**	Restricting direct access to data using private (__var) and protected (_var) modifiers.
Q15	**Difference between break and continue?**	break exits the loop; continue skips current iteration and goes to next.

Q16	What is a module?	A .py file containing functions, classes, variables that can be imported.
Q17	What is exception handling?	Using try-except to gracefully handle runtime errors instead of crashing.
Q18	What is the with statement?	Context manager that auto-closes resources (files) after the block ends.
Q19	What is the difference between / and //?	/ is true division (returns float); // is floor division (returns int).
Q20	What is None?	Represents absence of value. Similar to null in other languages. Type: NoneType.
Q21	What is a set?	Unordered collection of unique items. Supports set operations (union, intersection).
Q22	What is slicing?	Extracting a portion of sequence: s[start:stop:step]. Stop is excluded.
Q23	What is recursion?	Function calling itself. Must have a base case to avoid infinite recursion.
Q24	What is the difference between append and extend?	append() adds single element; extend() adds all elements from an iterable.
Q25	What is a generator?	Function with yield keyword. Returns values lazily (one at a time), saving memory.
Q26	What is a decorator?	Function that wraps another function to add functionality. Uses @decorator syntax.
Q27	Difference between local and global variable?	Local: defined inside function. Global: defined outside. Use global keyword to modify global inside function.
Q28	What is type casting?	Converting data from one type to another: int(), float(), str(), list(), etc.
Q29	What does enumerate() do?	Returns (index, value) pairs from an iterable. Avoids using a counter variable.
Q30	What is the difference between del and remove()?	del removes by index or deletes variable. list.remove() removes by value.

13. QUICK REVISION CHEAT SHEET

Syntax Quick Reference

Concept	Syntax
Variable	x = value
If-elif-else	if cond:\n ...\nelif cond2:\n ...\nelse:\n ...
For loop	for i in range(n): / for x in list:
While loop	while condition:
Function	def name(param, default=val): return value
Lambda	f = lambda x: x * 2
List comp.	[x**2 for x in range(10) if x % 2 == 0]
Dict comp.	{k:v for k,v in items.items()}
Class	class Name(Parent):\n def __init__(self, ...):
Try-except	try:\n ...\nexcept ExType as e:\n ...\nfinally:\n ...
File read	with open("f.txt","r") as f: data = f.read()
Import	import module / from module import name
List ops	append, insert, remove, pop, sort, reverse, index, count
Dict ops	keys(), values(), items(), get(), update(), del d[k]
String ops	upper, lower, strip, split, join, replace, find, format

Most Common Mistakes to Avoid

COMMON MISTAKES	BEST PRACTICES
Using = instead of == in conditions	Use f-strings for string formatting
Off-by-one errors in range() (stop is exclusive)	Use with for file operations
Forgetting : after if/for/def/class	Use enumerate() instead of range(len(lst))
Wrong indentation (Python is indent-sensitive)	Use list comprehension for simple transformations
Modifying list while iterating over it	Name variables descriptively
Using mutable default arg: def f(x=[])	Write docstrings for functions

Python vs Other Languages — Key Differences

Feature	Python	Java / C++
Type system	Dynamic (inferred)	Static (declared)
Syntax	Indentation-based	Braces {}
Variable declare	x = 5	int x = 5;
String type	str (built-in)	String class
Null value	None	null
Entry point	Any line / script	main() function
Memory mgmt	Automatic (GC)	Manual / GC

VIVA RAPID-FIRE ANSWERS

Python file extension: .py | Python interpreter: CPython (default)
Indentation size: 4 spaces (PEP 8) | Case sensitive: YES
Multiline string: triple quotes (''' or """) | Line continuation: \
len([]) = 0 | bool([]) = False | bool([0]) = True | type(None) = NoneType
Mutable: list, dict, set | Immutable: int, float, str, tuple, bool, frozenset
range() is lazy (generator-like) | list(range(5)) = [0,1,2,3,4]
Global keyword: use inside function to modify global variable
pass = empty placeholder | ellipsis (...) also used as placeholder
Python supports multiple inheritance: class C(A, B):
id() returns memory address | is checks if same object (same id)

QUICK SUMMARY

- Python is easy to learn but powerful — ideal for rapid development
- Master the 4 collections: list (mutable), tuple (immutable), set (unique), dict (key-value)
- OOP pillars: Encapsulation, Inheritance, Polymorphism, Abstraction
- Always handle exceptions, close files with with, and test your code
- GOOD LUCK IN YOUR EXAM! — AlgoForce AI